

從把程式寫出來，到讓一套系統真正運作

中文自傳 葉陽甫 | 國立花蓮高級中學

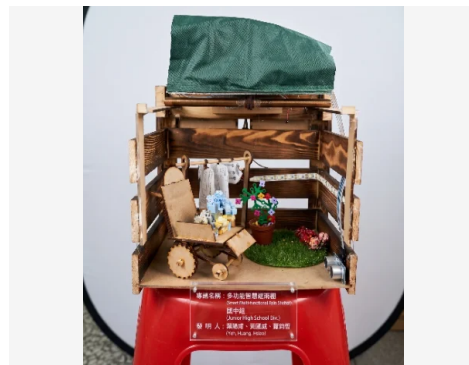
01 / 05



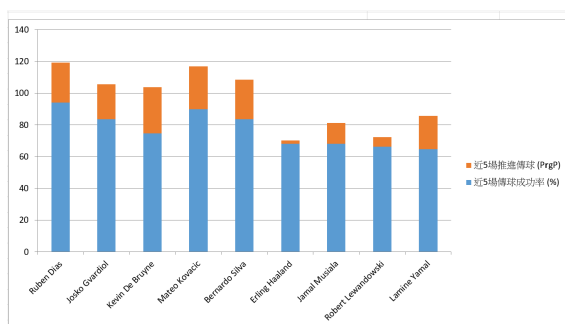
國中：第一次把程式接到實體世界

國中時，我和兩位同學做了一套多功能智慧遮雨棚，整合多種感測器與自動控制，App 控制介面由我獨立開發，作品獲得 IEYI 世界青少年發明展臺灣選拔賽銀牌。

這段經驗第一次讓我把程式、感測器與實體系統接在一起，也第一次看到程式在真實環境裡出錯。



國中發明展：第一次把程式、感測器與實體系統整合。



爬蟲抓下 FBref 數據後，整理成的球員戰術儀表板：近 5 場傳球成功率與推進傳球。

高一：IP 被封的那一次

高一時，我為了和朋友爭論球員近況，開始寫爬蟲抓 FBref (足球數據網站) 的球員資料。單一球員很快就成功，但連續送出大量請求後 IP 被封。我先換 VPN，結果新 IP 也被封，才回頭讀網站規範、理解 rate limiting，重新調整請求頻率與間隔。

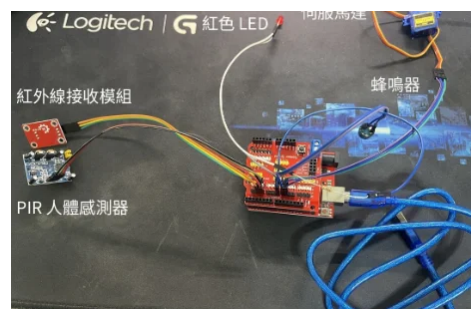
那次我才發現，程式其實寫得出來；真正卡住我的，是我還不了解正在使用的系統。

一路做下來，我學到的，是遇到錯誤時先把問題拆開，再慢慢理解系統怎麼運作。

高二：先拆開，再逐步整合

高二做 Arduino 防呆警報器時，模擬器一直讀不到紅外線訊號。我改用實體 Arduino，先單獨驗證接收器，再依序加入人體感測器、蜂鳴器與伺服馬達。

從那之後，我養成一個習慣：先拆開、逐一驗證，再整合。

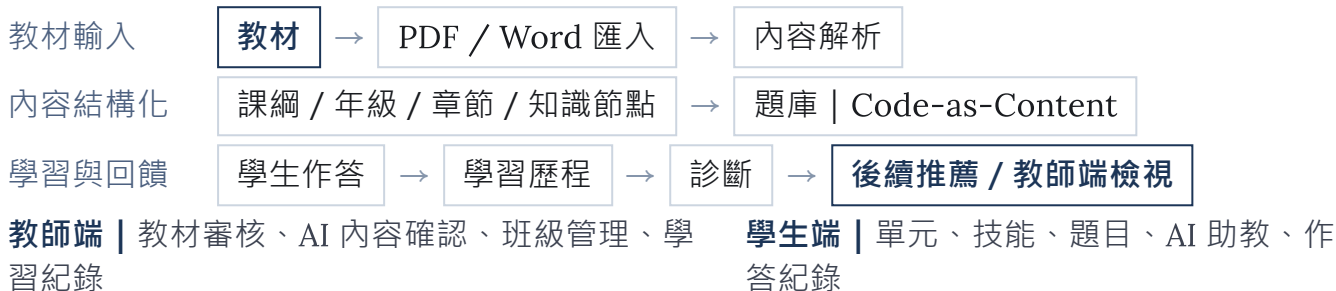


防呆警報器：逐一驗證每個模組後才整合。

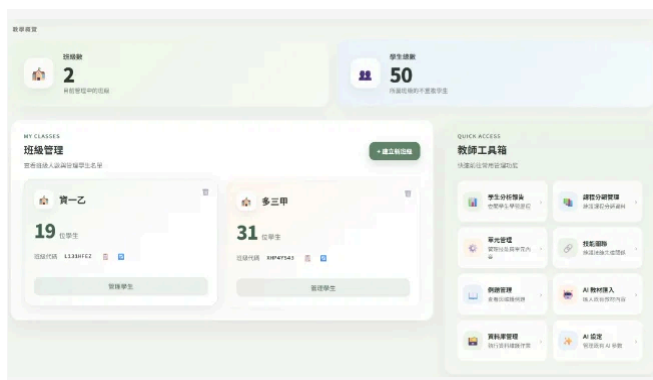
從單一功能，走向真正的大型 AI 學習系統

高二時，我主動向指導老師趙義雄老師詢問育秀盃的題目方向。想到父親在高職教數學時，常面對學生程度差異很大的問題，我和兩位同學因此開始做 AI 自適應學習系統。整套系統從教材匯入、內容解析、題庫生成，一路連到學生作答、學習紀錄、診斷與後續推薦，前後形成一條完整的資料鏈。

系統架構與我負責的資料主線 多角色 × 多模組 × 長資料鏈 × 長期維護



葉陽甫負責：從教材進入系統，到學生作答回傳的整條資料主線。



教師端：班級管理與 AI 教材匯入。

學生端：單元、技能與練習狀態。

延伸資料 | [系統實作影片：考卷診斷流程](#)

延伸資料 | [學生端展示：AI 即時救援](#)

大型專案難在哪裡

整套系統從教材匯入、內容解析、題庫生成，一路連到學生作答、學習紀錄、診斷與後續推薦，前後形成完整的資料鏈。任何前段欄位、格式或模組變動，都可能讓後續流程一起失效。

這套系統已經接受外部評審檢驗

系統後來獲得神通 AI 數位學院扶輪盃全國第一名，以及育秀盃高中職 AI 應用類金獎。對我而言，更重要的是：這套從真實教學問題出發、整合教材、題庫、作答與診斷流程的系統，第一次接受了團隊以外的外部檢驗。

得獎之後，系統還繼續往下做，我也開始碰到成本、延遲、錯誤與維運問題。這些問題後來逐漸改變了我的方向。



大型專案開始真正運作後，問題才出現



Code-as-Content 的代價

題庫採用 Code-as-Content：把出題程式存進資料庫，由 Python 即時生成變題與答案。這讓一支程式可以反覆產生大量變題，但系統長期運作後，我開始看到另一個問題：每次都重新呼叫大型模型生成程式，token、API 成本與回應時間會持續累積；而同一數學單元中，許多程式結構與錯誤其實會反覆出現。

延伸資料 | [教材匯入與建立](#) ↗

已經解決過的問題，有沒有必要每一次都重新交給大型模型思考？



Scaffolding 解決的是「不要讓模型每次都從零規劃」。

大模型先把工具、步驟與資料流整理清楚，縮小小模型需要自行規劃的範圍。

AST Healer 解決的是「不要每次局部錯誤發生後都讓模型重新生成」。

只有能由固定規格唯一判定的局部錯誤才修；需要重新理解題意時，才交回大型模型。

Edge Model 負責已經熟悉的重複生成，遇到需要重新理解題意的問題，再交回大型模型。

這些實際維運時碰到的問題，後來成了我研究 Scaffolding 與 AST Healer 的起點。

從漂亮分數，到知道什麼才算被證明

初版消融實驗在自創指標 (MCRI) 分數達到 97.41，我一度以為方法已經得到充分驗證。這份研究後來在東區科展獲得電腦與資訊學科優等，但沒有晉級全國。這讓我開始回頭檢查：問題到底出在哪裡？

到了旺宏科學獎，評審的問題又讓我進一步想到：這個分數是在什麼條件下得到的？這套評分又能證明什麼？我重新拆解實驗流程，才發現測試題來自自己整理的規則，我的指標也是為這套系統設計的指標。97.41 沒有算錯，但它只能證明方法在當時的測試條件與評分框架下表現良好。

回頭檢查後，我發現 97.41 的計算本身沒有錯，真正的問題出在開發與評估沒有切開。測試資料同時參與了規則調整與效果判斷，使高分可能同時反映方法本身，也反映我對測試條件的熟悉。

我後來才知道，分數變高，並不代表方法已經被充分驗證。

重新設計驗證

重新驗證時，我把問題改成：哪一個環節真的讓生成變可靠？原本失敗的案例，又有多少能被安全救回？

改用公開會考試題

開發 / 評估分開

Healer 看不到正確答案

自訂分數 → 救援率

Scaffolding 的發現

我原本以為，提供更多 API 與規格會讓模型表現更好，但第一次比較時卻發現，只增加 API 資訊，Qwen 4B 的通過率從 23.3% 降到 8.3%，Qwen 9B 也從 30.0% 降到 21.7%。這讓我開始懷疑，問題可能不在資訊夠不夠多，而在模型能不能把這些資訊組織成正確流程。

後來在相同完整 API 條件下，我進一步把必要工具、參數來源與處理順序整理成完整流程提示，Qwen 4B 與 9B 的表現分別提高 47.50 與 23.75 個百分點。這讓我真正理解 Scaffolding 的作用：把流程整理清楚，降低模型自行規劃的負擔。

初版消融實驗 · MCRI
(自創指標 Math Code Reliability Index)

41.00 → 94.39 → **97.41**

基準 → 加入 Scaffolding → 再加入 AST Healer

Held-out / answer-blind 驗證
· safe rescue rate

2.63%

342 個原始失敗 → 介入 42 個 → 安全救回 9 個

Healer 只修能由固定規格安全判定的局部錯誤；涉及答案或主要演算法時，則不介入。

驗證方法 | 16 題公開會考題中，4 題用於規則開發，另外 12 題到規則固定後才正式評估。

[旺宏科學獎完整成果報告 ↗](#)

為什麼是資訊工程，為什麼是清大資工

這次研究改變了我看待結果的方式

2.63% 比我原本期待的低很多，但因為這次把開發與評估真正分開，我反而比較相信這個結果。這次重新驗證也讓我**知道**，之後做研究時，我必須先問清楚一個結果到底能證明什麼。

Scaffolding 的實驗讓我開始關心模型如何被工具、介面與流程約束；AST Healer 則讓我開始關心程式分析、軟體可靠度與安全修復的邊界。

從硬體、爬蟲到 AI 學習系統與程式修復，我反覆追問的都是：系統為什麼會失敗、問題能不能被準確找出來，以及怎麼證明自己真的修好了。這些問題我都是在專案裡撞到才學會的，但我很清楚，光靠專案拼起來的知識撐不到下一步。

我要補的基礎

資料結構、離散數學、演算法、計算機組織、作業系統。

我想深入的方向

程式分析、生成可靠性、AI 系統設計、大型軟體系統。

為什麼是清大資工

我希望把靠專案累積的實作經驗，接回資料結構、演算法、計算機組織與作業系統等完整基礎，再往程式分析、軟體可靠度與 AI 系統設計深入。我想把目前在大型專案與 AST Healer 中碰到的問題，帶進更完整的資訊工程訓練裡繼續追下去。

我目前的位置

APCS 程式識讀 92 分・第五級 / 程式實作 135 分・第二級

這個落差誠實反映我的位置：讀得懂、拆得開，但演算法實作仍需要正式訓練，這就是我進大學後最需要補強的一塊。

下一步：真正進入教室的驗證

系統下一步將進入父親任教班級與學校 AI 實驗班，共 50 位學生。下一步，我想進一步確認：學生的成績如果出現變化，究竟有多少能歸因於這套系統。

前面的實驗已經讓我**知道**，看到分數上升還不夠。等系統真的進入教室後，我想把「學生有沒有進步」和「是不是系統造成的」分開驗證。